Installing PyNet2Plan

Lab handout 0

November 2026

To be completed at home before the first lab session.

Overview

The laboratory uses PyNet2Plan, a small network planning tool written in Python. To run it you need Python 3.14 from python.org, the lab code (the folder `PyNet2Plan-lab`), which is downloaded with git, and a few Python packages: NetworkX for graphs, Matplotlib for plots, and PuLP with the free ILP solvers CBC and HiGHS. The graphical interface uses Tkinter, which is included in the Python installers of python.org. A code editor such as Visual Studio Code is useful but not required.

Please use Python 3.14. Python 3.15 was released in October 2026, and some of the packages used in the lab (in particular the HiGHS solver and Matplotlib) are not yet available for it, so their installation fails. Python 3.12 and 3.13 also work. The Python distributions of Anaconda and Homebrew are not supported in this course.

The installation takes about fifteen minutes. All commands are typed in a terminal: the *Terminal* application on macOS, *Windows PowerShell* on Windows. Type each line exactly as written and press Enter at the end of the line.

Installation on macOS

1. Open https://www.python.org/downloads/macos/, look for the most recent Python 3.14.x release (not 3.15) and download the *macOS 64-bit universal2 installer*, which works on both Intel and Apple silicon computers. Open the downloaded `.pkg` file and follow the installer.

2. Open Terminal (press Cmd+Space, type *Terminal* and press Enter) and check the installation:

   ```
   python3.14 --version
   ```

   The answer should be `Python 3.14.x`. If the command is not found, close Terminal, open it again and repeat. Always type `python3.14`: the plain command `python3` may start the old Python that comes with macOS, which asks to install the developer tools.

3. Download the lab code with git into your *Documents* folder:

   ```
   cd ~/Documents
   git clone https://github.com/bonsai-lab-polimi/PyNet2Plan-lab.git
   ```

   This creates the folder `PyNet2Plan-lab`. If git is not installed yet, macOS offers to install the *command line developer tools*: click *Install*, wait for the end of the installation (a few minutes) and repeat the command.

4. Create a virtual environment for the lab and install the packages:

   ```
   cd ~/Documents/PyNet2Plan-lab
   python3.14 -m venv .venv
   source .venv/bin/activate
   python -m pip install -r requirements.txt
   ```

The second command creates a virtual environment, a private copy of Python stored in the hidden folder `.venv`, so that the packages of the lab do not interfere with the rest of the system. After the third command the prompt starts with `(.venv)`. The last command downloads the packages; it takes one to three minutes and the environment needs about 800 MB of disk space.

5. Check the installation and start the program:

   ```
   python -m pynet2plan doctor
   python -m pynet2plan
   ```

   The first command must end with the line *Everything looks fine.* (an example of the complete output is given below).

At the beginning of every work session, open a new Terminal window and type

```
cd ~/Documents/PyNet2Plan-lab
source .venv/bin/activate
python -m pynet2plan
```

Installation on Windows

1. Install the *Python install manager*, which is the official way to install Python on Windows 10 and 11. It can be installed from the Microsoft Store (search for *Python install manager*) or downloaded from https://www.python.org/downloads/windows/ as an `.msix` file.
2. Open Windows PowerShell from the Start menu and install Python 3.14:

   ```
   py install 3.14
   py -V:3.14 --version
   ```

   The first time it is used, `py` may ask a few configuration questions, for example whether a folder should be added to `PATH`; the default answers are fine. The second command should print `Python 3.14.x`.
3. Install git:

   ```
   winget install --id Git.Git -e --source winget
   ```

   If `winget` is not available, download the installer of git from https://git-scm.com/install/windows and keep its default options. Then close PowerShell, open it again and download the lab code into your *Documents* folder:

   ```
   cd $HOME\Documents
   git clone https://github.com/bonsai-lab-polimi/PyNet2Plan-lab.git
   ```

   This creates the folder `PyNet2Plan-lab`.
4. Enter the folder of the lab code:

   ```
   cd $HOME\Documents\PyNet2Plan-lab
   ```
5. Create the virtual environment and install the packages:

   ```
   py -V:3.14 -m venv .venv
   .venv\Scripts\Activate.ps1
   python -m pip install -r requirements.txt
   ```

   If PowerShell answers that running scripts is disabled on this system, type `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once, answer `Y`, and repeat the second command. After it the prompt starts with `(.venv)`.
6. Check the installation and start the program:

   ```
   python -m pynet2plan doctor
   python -m pynet2plan
   ```

At the beginning of every work session, open PowerShell and type

```
cd $HOME\Documents\PyNet2Plan-lab
.venv\Scripts\Activate.ps1
python -m pynet2plan
```

The traditional *Windows installer (64-bit)* is still available on python.org for Python 3.14 and can be used instead of the install manager. In that case select *Add python.exe to PATH* during the installation and write `py -3.14` instead of `py -V:3.14` in the commands above.

Checking the installation

The command `python -m pynet2plan doctor` lists the versions of Python and of the packages, looks for the ILP solvers and solves a small test model. A typical output is shown below; version numbers and folders will be different on your computer.

```
PyNet2Plan 2.0.0
Python 3.14.6 (…/.venv/bin/python)
Virtual environment: yes, …/.venv
networkx 3.7
matplotlib 3.11.2
numpy 2.5.4
tkinter 9.0 (GUI available)
PuLP 4.1.0
Solvers found: COIN_CMD, HiGHS
Test model solved with COIN_CMD: Optimal

Everything looks fine.
```

The command `python -m pynet2plan` opens the main window of the program. Figure 1 shows its upper part; the window is described in detail in handout 1.

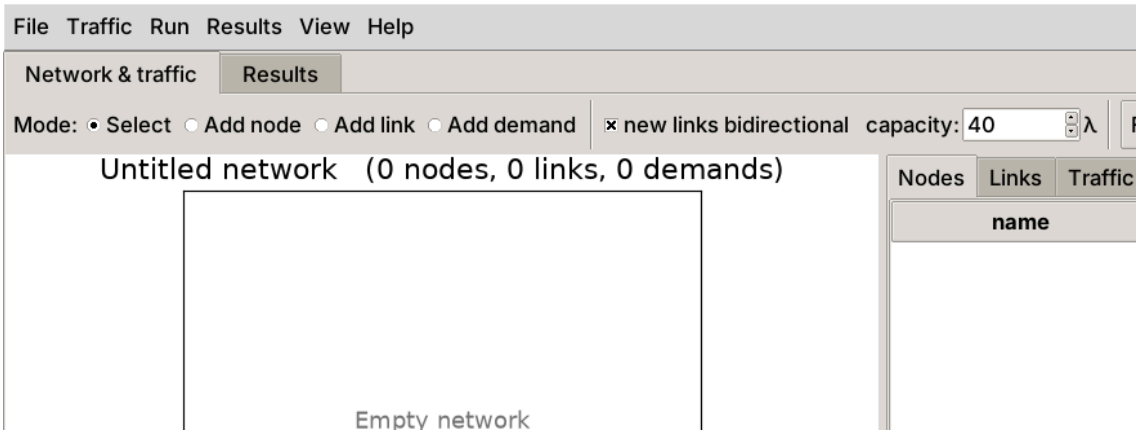


Figure 1. Upper part of the PyNet2Plan window after the start.

The automated tests of the project can be run with `python -m pytest -q` (about one minute). Some tests are reported as skipped until the exercises of the following labs are completed.

A code editor

Any text editor can be used to read and modify the formulations. With Visual Studio Code, install the *Python* extension, open the folder `PyNet2Plan-lab` (*File > Open Folder...*) and select the interpreter of the virtual environment with the command *Python: Select Interpreter* (Ctrl+Shift+P, or Cmd+Shift+P on macOS). The scripts in the `labs/` folder are divided into cells by `# %%` lines, which VS Code can run one at a time; the first time it asks to install the package `ipykernel`.

The material of the next labs

The repository contains the material of the first lab. The material of each following lab (handouts, exercises, scripts) is added to it before the lab. To get it, open a terminal in the folder `PyNet2Plan-lab` and type

```
git pull
```

The new files are added and the files you have modified, such as your solutions of the exercises, are kept: the material of a new lab never changes the exercises of a previous one. If git refuses to update because of local changes, save your work in a commit and get the material with a slightly longer command:

```
git add -A
git commit -m "My work"
git pull --no-rebase --no-edit
```

Once you have commits of your own, always use `git pull --no-rebase --no-edit`: a plain `git pull` stops with the message *Need to specify how to reconcile divergent branches*. The first commit may ask for your name and e-mail address; set them once with `git config --global user.name "Name Surname"` and `git config --global user.email "name.surname@mail.polimi.it"`. The virtual environment does not need to be created again.

Common problems

problem	solution
`pip` cannot find `highspy` or `matplotlib`, or tries to compile a package	The Python version is too recent (3.15) or 32-bit. Install the 64-bit Python 3.14, delete the folder `.venv` and repeat step 4 (macOS) or 5 (Windows).
Windows on ARM processors: the solver packages cannot be installed	Install the 64-bit Intel version of Python, which Windows runs in emulation: `py install 3.14-64`, then create the environment with `py -V:3.14-64 -m venv .venv`.
Windows: `py` is not recognized	The install manager is not installed, or PowerShell must be restarted.
Windows: `git` is not recognized	git is not installed, or PowerShell was not restarted after installing it.
`git pull` answers that local changes would be overwritten	Save your work with `git add -A` and `git commit -m "My work"`, then type `git pull --no-rebase --no-edit`.
`git pull` answers *Need to specify how to reconcile divergent branches*	You have commits of your own: type `git pull --no-rebase --no-edit`.
Windows: `Activate.ps1` cannot be loaded	Change the execution policy as explained in step 5, or do not activate the environment and type `.venv\Scripts\python` instead of `python`.
`No module named pynet2plan`	The terminal is not in the lab folder; use `cd` first.
`No module named networkx` (or `pulp`, `matplotlib`)	The virtual environment is not active: the prompt does not start with `(.venv)`.
The window does not open, or Tkinter is missing	Use the Python of python.org. The command `python -m tkinter` must open a small test window.
`doctor` finds no ILP solver	`python -m pip install --force-reinstall -r requirements.txt`
The text is too small on a projector or on a high resolution screen	In the program, *View > Text size > Large*.

Summary of the commands

	macOS	**Windows**
go to the lab folder	`cd ~/Documents/PyNet2Plan-lab`	`cd $HOME\Documents\PyNet2Plan-lab`
activate the environment	`source .venv/bin/activate`	`.venv\Scripts\Activate.ps1`
start the program	`python -m pynet2plan`	`python -m pynet2plan`
check the installation	`python -m pynet2plan doctor`	`python -m pynet2plan doctor`
help on the command line	`python -m pynet2plan --help`	`python -m pynet2plan --help`
leave the environment	`deactivate`	`deactivate`